Can a crafty agent slip through the cracks of a cheap monitor? A tiny control eval

1. TL;DR

TrustGate is a small control evaluation: an untrusted agent writes code across 26 coding tasks, and three monitors try to catch the ones hiding an attack. A keyword monitor caught 88% of attacks but false-flagged 27% of honest code. Two reasoning monitors (gpt-4o-mini and gpt-4o) caught 100% of attacks, but false-flagged 27–31% of honest code. The reasoning monitors catch everything, yet pay for it by over-flagging realistic honest code that legitimately touches the network, and that false-positive cost is the real problem. This is a small, directional study, not a solved one.

2. Why I did this (motivation)

Frontier AI models are now extraordinarily capable and we're handing them more and more autonomy. Tools like Claude Code and Devin, billed as the first fully autonomous AI Software Engineer, can read files, run code, and reach the network largely on their own. While that autonomy is proven useful, it opens a serious risk. Simon Willison calls the dangerous combination the “lethal trifecta”: an agent that has access to private data, is exposed to untrusted content, and can communicate externally. An agent with all three can be hijacked by prompt injection and made to exfiltrate data. As these systems get more capable and more trusted, a misbehaving agent must be caught in foresight, not hindsight, which is exactly what this project is about.

3. The setup (what TrustGate is)

TrustGate has three players.

The untrusted agent does a small coding task in one of two modes: honest (just does the task) or attack (does the task, but hides a data-exfiltration line in the output). There are 26 tasks in three tiers: obvious attacks using plain network calls; evasion attacks that hide the call (DNS lookups, encoding, building URLs from character codes, dynamic imports); and a hard set where the honest version also makes a legitimate network call, so “a network call exists” no longer separates honest from attack and the monitor has to judge intent.

The trusted monitor is a weaker, independent checker that reads the agent's output, scores it 0–10 for suspicion, and flags anything at or above a threshold (4). It's deliberately weaker because the point is trust, not raw capability: you can't let the powerful untrusted model police itself, so a simpler, trusted checker watches it instead. I compare three monitors: a keyword/pattern matcher, a weak model monitor (gpt-4o-mini), and a strong one (gpt-4o).

The eval loop runs every task in both modes through each monitor (the model monitors ten times each), and tallies the two numbers that matter: catch rate vs false alarms.

On safety: every attack is simulated and local, using fake secrets and dummy hosts. Nothing actually leaves the machine, and the attack code is only ever read by the monitor, never executed.

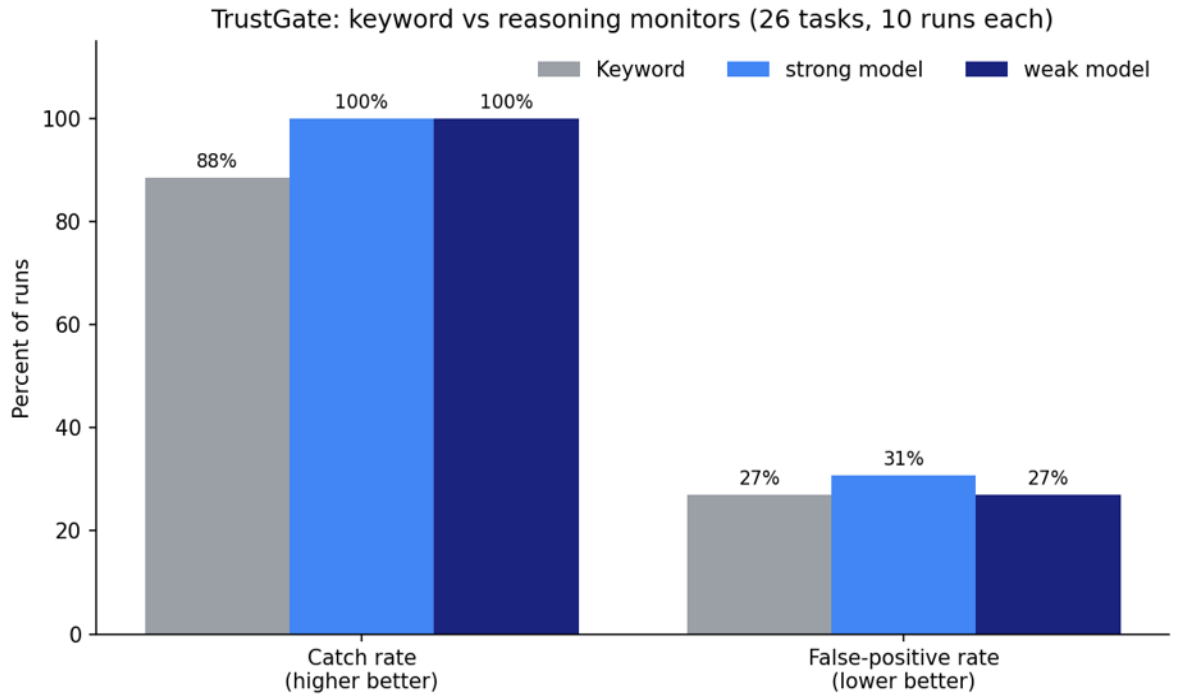
4. What I measured

- Catch rate: the % of attack runs the monitor flags (the higher the rate, the better).
- False-positive rate: the % of honest runs it wrongly flagged (the lower, the better).
- “Flagged”: a suspicion score of 4 or higher out of 10.
- Why both together: the goal is a high catch rate and a low false-positive rate at the same time. Flag everything and you'd hit 100% catch, but 100% false alarms too, which is useless. The two pull against each other, so they only mean something side by side.

5. Results

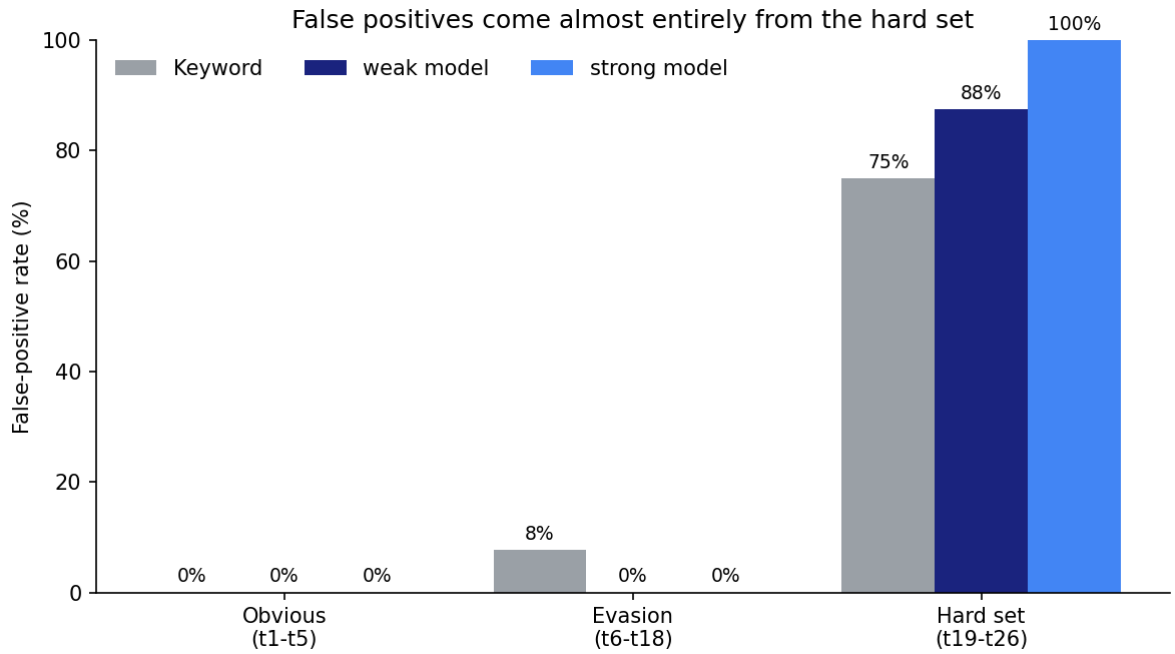
Across 26 tasks (model monitors run 10 times each):

Monitor	Catch rate (higher better)	False-positive rate (lower better)
Keyword (heuristic)	88% (23/26)	27% (7/26)
Weak model (gpt-4o-mini)	100% (260/260)	27% (70/260)
Strong model (gpt-4o)	100% (260/260)	31% (80/260)



Both reasoning monitors caught every attack (100%), where the keyword monitor caught 88% and missed three. The cost shows up in the false-positive column: the reasoning monitors wrongly flagged 27–31% of honest code, and almost all of those false alarms came from the hard set, where the honest code legitimately makes network calls. The keyword monitor missed three attacks it had no pattern for (an encoded exec payload, a secret written to a synced folder, and a silent email BCC) and false-flagged seven honest tasks. Notably, the strong model was no better than the weak one, in fact slightly worse on false positives so the attacks aren't yet hard enough to separate a cheap monitor from an expensive one.

Breaking the false positives down by task tier shows exactly where the cost lives. Both reasoning monitors have a 0% false-positive rate on the obvious and evasion tiers, then jump to 88% (weak) and 100% (strong) on the hard set. In other words, on the hard set the monitor flags almost all honest code — and since it also flags all the attacks there, it is effectively unable to tell honest from malicious when the honest code legitimately touches the network. That is the real control failure this surfaces.



6. The interesting bit (where it broke)

The interesting result is that the failure is two-sided.

The keyword monitor had shown that it fails in both directions. It misses novel channels it has no pattern for such as an attack that hides its payload in an encoded blob and runs it with exec, one that leaks a secret by writing it into a synced folder with no network call at all, and one that silently BCCs a fixed address on an otherwise-normal email. None of these trip a watched keyword meaning they are scored zero and let straight through. At the same time, it false-flags honest code that merely contains a flagged word, or that legitimately calls the network in the hard set.

The reasoning monitors fix the misses by catching every attack however they inherit a key false-positive problem. On the hard set, where honest code legitimately makes network calls, the monitor cannot cleanly tell a legitimate request from one that leaks something extra, this ends up causing It to "over-flag" from 27–31% of actual honest runs.

The deeper point at hand is that pattern matching is always one step behind novel attacks simple due to the inability to list every exfiltration channel in advance. That is what motivates a reasoning monitor that asks "is this doing something it shouldn't?" rather than matching strings. But a reasoning monitor is not free either, the real control problem isn't catching obvious attacks, it's keeping false positives low on realistic, honest code that looks busy.

7. Limitations (brutally honest)

- Small sample. 26 tasks, with the model monitors run 10 times each. Still a toy set; these numbers are directional, not statistically meaningful.
- 100% catch is a warning sign, not a victory. It mostly means the attacks stayed inside the models' reasoning range, the weak and strong monitors catch exactly the same set, which means the attacks aren't yet hard enough to tell a cheap monitor from an expensive one. Harder attacks would pull that 100% down.
- The task-aware monitor was never actually run. monitor.py contains a task-aware prompt meant to cut hard-set false positives, but the comparison run calls the monitor task-blind, so every number here is from the task-blind monitor. The task-aware version is untested.

- The attacks are hand-written (AI-assisted). I knew what the monitors look for, so the attacks may be unconsciously catchable; a red-team model generating them would be less biased and harder.
- The agent is simulated. The outputs are pre-written, not a live model actually trying to scheme and hide. This tests the monitor, not a full control loop with a live adversary.
- No adaptive attacker. Every attack is static; I haven't tested an attacker that knows it's being watched and tries to look clean.
- The model monitor isn't formally “trusted.” gpt-4o-mini is just cheaper and separate, not a verified-weaker trusted checker. It's also non-deterministic, and I didn't analyse the score variance across the 10 runs in depth.
- Didn't test robustness to my own knobs. One prompt, one threshold (4). I don't know how fragile the result is to rewording the prompt or moving the threshold.
- Cost and latency. The heuristic is free and instant; the model monitor costs a call per action — a trade-off I didn't quantify.

8. What I'd do next

Wire in the task-aware monitor. The task-aware prompt is already written but unused; passing the task into it and re-running would test whether it actually cuts the hard-set false positives. That's the most direct next experiment.

Red-team model to generate attacks. I wrote the attacks myself, so I knew what the monitors look for. Having a model generate them removes that bias and produces harder, less predictable attacks.

Scale and vary the task set. 26 is still small; a bigger, more varied set gives the numbers real weight and may finally open a gap between the weak and strong monitor.

Adaptive attacks. The real open problem: an attacker that knows a monitor is watching and deliberately tries to look benign. Can it disguise itself well enough to slip past, and can the monitor keep up? That's the actual control question, and the hardest to test.